

Greedy O(N) Solution

Dry Run of Greedy O(N) Solution

Example

```
s = "adefaddacc"
```

Index Representation

```
Index : 0 1 2 3 4 5 6 7 8 9 10
Char  : a d e f a d d a c c c
```

Step 1: Compute first[] and last[]

We scan the string once.

Character	First Occurrence	Last Occurrence
a	0	7
d	1	6
e	2	2
f	3	3
c	8	10

(All other characters never appear.)

Step 2: Initialize Variables

```
C++
ans = { }
prevEnd = -1
```

Now iterate over every index.

i = 0

Character

```
a
```

Is this the first occurrence?

```
first[a] = 0
```

Yes.

So try building an interval starting here.

Initial Interval

```
start = 0
end = last[a] = 7
```

Current interval

```
[0.....7]
```

Now expand this interval.

j = 0

Character

```
a
```

Check

```
first[a] < start ?
0 < 0
```

No.

Expand

```
end = max(7,7) = 7
```

No change.

j = 1

Character

```
d
```

Check

```
first[d] = 1
1 < 0 ?
```

No.

Expand

```
end = max(7,6) = 7
```

No change.

j = 2

Character

```
e
```

Check

```
first[e] = 2
2 < 0 ?
```

No.

Expand

```
end = max(7,2) = 7
```

j = 3

Character

```
f
```

Check

```
first[f] = 3
3 < 0 ?
```

No.

Expand

```
end = max(7,3) = 7
```

j = 4

Character

```
a
```

Already inside the interval.

Nothing changes.

j = 5

Character

```
d
```

Already covered.

j = 6

Character

```
d
```

Already covered.

j = 7

Character

```
a
```

Already covered.

Final Interval

```
[0.....7]
```

Substring

```
"adefadda"
```

Greedy Selection

Check

```
C++
i > prevEnd
0 > -1
```

Yes.

Push it.

```
ans = { "adefadda" }
prevEnd = 7
```

i = 1

Character

```
d
```

Is it the first occurrence?

```
first[d] = 1
```

Yes.

Try building its interval.

Initial Interval

```
start = 1  
end = 6
```

Scan from 1 to 6.

j = 1

Character

```
d
```

Everything is fine.

j = 2

Character

```
e
```

Everything is fine.

j = 3

Character

```
f
```

Everything is fine.

j = 4

Character

```
a
```

Now check

```
first[a] = 0  
0 < start(1)
```

Yes.

So

```
C++  
valid = false;  
break;
```

Why?

Our interval starts at index 1.

But it contains character **a**, whose first occurrence is at **0**.

That means this interval cannot contain **all** occurrences of **a**.

So this interval is **invalid**.

Reject it.

i = 2

Character

```
e
```

Is this its first occurrence?

Yes.

Initial Interval

```
start = 2
end = 2
```

Only one character.

Completely valid.

Substring

```
"e"
```

Greedy Decision

Current interval

```
[2,2]
```

Previously selected interval

```
[0.....7]
```

Notice

```
2 <= 7
```

They overlap.

The code executes

```
C++
ans.back() = "e";
```

Now

```
ans = { "e" }
prevEnd = 2
```

i = 3

Character

```
f
```

First occurrence?

Yes.

Interval

```
[3,3]
```

Valid.

Check

```
C++
3 > prevEnd
3 > 2
```

Yes.

Push it.

```
ans = { "e", "f" }
prevEnd = 3
```

i = 4

Character

```
a
```

Not the first occurrence.

Skip.

i = 5

Character

```
d
```

Not first.

Skip.

i = 6

Character

```
d
```

Skip.

i = 7

Character

```
a
```

Skip.

i = 8

Character

```
c
```

First occurrence?

Yes.

Initial Interval

```
start = 8
end = 10
```

Scan

```
8
9
10
```

Only character c.

Valid.

Substring

```
"ccc"
```

Greedy Decision

Check

```
C++
8 > prevEnd
8 > 3
```

Yes.

Push.

```
ans = { "e", "f", "ccc" }
prevEnd = 10
```

Final Answer

```
["e", "f", "ccc"]
```

The Most Important Greedy Step

This is the part that confuses almost everyone.

```
C++
if (i > prevEnd)
{
    ans.push_back(s.substr(i, end - i + 1));
}
else
{
    ans.back() = s.substr(i, end - i + 1);
}
```

Let's understand it visually.

Initially

We selected

```
ade fadda
0-----7
```

Later we discovered

```
e
  2
```

The intervals look like

```
0-----7
  2
```

The second interval is **completely inside** the first one.

Both intervals overlap.

The first interval ends at 7.

The second interval ends at 2.

Clearly,

```
2 < 7
```

So the second interval finishes much earlier.

Therefore we replace

```
ade fadda
```

with

```
e
```

Instead of keeping

```
0-----7
```

we keep

```
2
```

because it leaves much more unused space for future intervals.

Why is this always correct?

Suppose we keep the large interval.

```
0-----7
```

Everything between 0 and 7 is blocked.

Any future interval starting before 8 cannot be selected.

If instead we keep

```
2
```

then only index 2 is occupied.

Now intervals beginning at

```
3
4
5
6
7
8
...
```

still have a chance to be selected.

Therefore, **keeping the interval that ends earliest can never reduce the number of intervals we can choose later.**

In fact, it can only increase our opportunities.

Connection to Interval Scheduling

This is exactly the classic greedy strategy used in Interval Scheduling problems.

When two valid intervals overlap:

```
Large Interval
0-----7
Small Interval
  2
```

Always keep the one with the **smaller ending index**.

This leaves the maximum remaining space for future intervals.

Complete Greedy Flow

```
Initially
ans = { }
↓
Choose
adefadda
↓
Find smaller valid interval
e
↓
Replace previous interval
```



```
ans = { "e" }
↓
Find
f
↓
Push
ans = { "e", "f" }
↓
Find
ccc
↓
Push
ans = { "e", "f", "ccc" }
```

Final Intuition

The entire algorithm is based on two observations:

1. **A valid substring must contain every occurrence of every character inside it.**
While expanding an interval, if we encounter a character whose **first occurrence lies before the current start**, the interval is impossible and must be rejected.
2. **Among overlapping valid intervals, always keep the one that finishes earliest.**
A shorter ending interval leaves more room for future non-overlapping intervals, which maximizes the total number of substrings.

This second observation is the classic **Earliest Finish Time Greedy** strategy from interval scheduling, and it is the key reason this solution runs in **O(n)** time while still producing the optimal answer.

```
C++
/*
=====
LeetCode 1520 - Maximum Number of Non-Overlapping Substrings
Greedy O(N) Solution
=====
ALGORITHM
-----
1. Find the first and last occurrence of every character.

2. Iterate through the string.
   We only try to build an interval from the FIRST occurrence of a character.
   (Starting from any other occurrence can never produce the minimum interval.)

3. Build the interval.
   - Initially:
     start = current index
     end   = last occurrence of current character

   - Traverse from start to end.

     For every character inside the interval:

     A)
     If its first occurrence is BEFORE the current start,
     then this interval can never contain all occurrences of that character.

     Therefore:
       interval is INVALID.

     B)
     Otherwise,
     expand the interval to include the last occurrence of this character.

4. If the interval is valid:

   Case 1:
   -----
   It does NOT overlap with the previously selected interval.

       start > prevEnd

   Simply add it.

   Case 2:
   -----
   It overlaps.

   Since the current interval always ends earlier,
   replace the previous interval.

   Keeping the interval that ends earliest leaves more room
   for future intervals (classic interval scheduling greedy).

TIME COMPLEXITY
-----
Finding first/last occurrence : O(N)
```

```

Building intervals      : 0(26 * N)
                        = 0(N)

Greedy selection        : 0(26)

Overall                 : 0(N)

SPACE COMPLEXITY
-----
first[] + last[] = 0(26)

Overall = 0(1)

=====
*/

class Solution {
public:
    vector<string> maxNumOfSubstrings(string s) {
        int n = s.size();

        // first[i] = first occurrence of character i
        // last[i] = last occurrence of character i
        vector<int> first(26, n);
        vector<int> last(26, -1);

        //-----
        // Step 1 : Compute first and last occurrence of every character
        //-----
        for (int i = 0; i < n; i++) {
            int ch = s[i] - 'a';

            first[ch] = min(first[ch], i);
            last[ch] = i;
        }

        vector<string> ans;

        // Ending index of previously selected interval
        int prevEnd = -1;

        //-----
        // Step 2 : Try to build interval from every FIRST occurrence
        //-----
        for (int i = 0; i < n; i++) {
            int ch = s[i] - 'a';

            // Ignore if this is not the first occurrence.
            if (i != first[ch])
                continue;

            // Current interval
            int start = i;
            int end = last[ch];

            bool valid = true;

            //-----
            // Step 3 : Expand interval
            //-----
            for (int j = start; j <= end; j++) {
                int cur = s[j] - 'a';

                // This character started before our interval.
                // Therefore this interval cannot contain all occurrences.
                if (first[cur] < start) {
                    valid = false;
                    break;
                }

                // Expand interval to include all occurrences
                end = max(end, last[cur]);
            }

            // Invalid interval
            if (!valid)
                continue;

            //-----
            // Step 4 : Greedy Interval Scheduling
            //-----

            // No overlap with previous interval
            if (start > prevEnd) {
                ans.push_back(s.substr(start, end - start + 1));
            }
            else {
                // Overlap exists.
                // Current interval always ends earlier,
                // so replacing is always better.
                ans.back() = s.substr(start, end - start + 1);
            }

            prevEnd = end;
        }

        return ans;
    }
};

```

Key Observations to Remember

1. Why do we only start from the first occurrence?

- Any valid substring containing a character must include its first occurrence.
- Starting from a later occurrence can never produce the minimum valid interval.

2. Why do we expand the interval?

- Every character inside the interval must have **all** of its occurrences included.
- If a character appears later, we extend the interval to its last occurrence.

3. Why is `first[cur] < start` invalid?

- It means this character already appeared before our interval started.
- Since we didn't include that earlier occurrence, we cannot include **all** occurrences of that character.
- Hence the interval is invalid.

4. Why do we replace the previous interval when they overlap?

- The current valid interval always ends earlier.
- Keeping the interval with the earliest ending leaves the maximum remaining space for future intervals.
- This is the classic **Earliest Finish Time Greedy** strategy from interval scheduling.

This is the standard **O(N)** greedy solution accepted on LeetCode and is considered the optimal approach for this problem.
 ChatGPT can make mistakes. Check important info.